

배경

정적 분석(Static Analysis)은 프로그램을 실행하지 않고 코드의 실행 특성을 추론하여 잠재적인 오류를 탐지하는 기술이다. IDE, 컴파일러, CI 환경 등 다양한 개발 과정에서 코드 품질과 안정성을 높이기 위해 널리 활용되고 있음.
기존 분석기들은 최종 경고 결과만 제시할 뿐, 어떤 과정을 통해 해당 결과가 도출되었는지는 충분히 보여주지 않음. 이로 인해 사용자는 분석 결과가 왜 발생했는지 이해하기 어렵고, 분석 과정을 일종의 “블랙박스”처럼 받아들이고 결과를 맹신하게 됨.

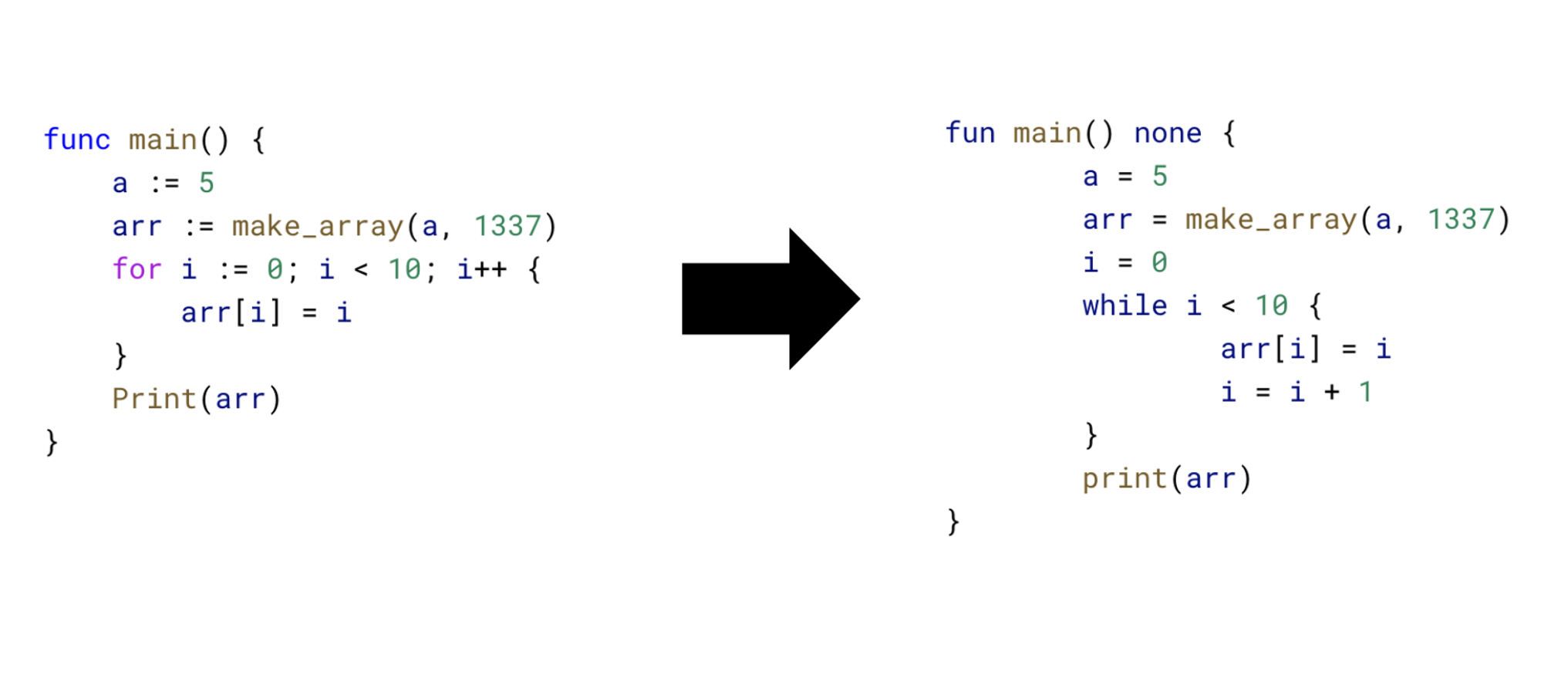
TraceInspector 소개

TraceInspector는 요약해석(Abstract Interpretation) 이론을 바탕으로 한 정적 분석기와 더불어 분석 과정을 시각적으로 보여주는 인터페이스로 구성되어 있음.
분석 중 생성되는 요약 상태(Abstract State)의 변화 과정을 사용자에게 실시간으로 보여주어 각 프로그램 지점에서 정보가 어떻게 전파 및 정제되는지를 직관적으로 관찰할 수 있도록 구현함.

분석 과정

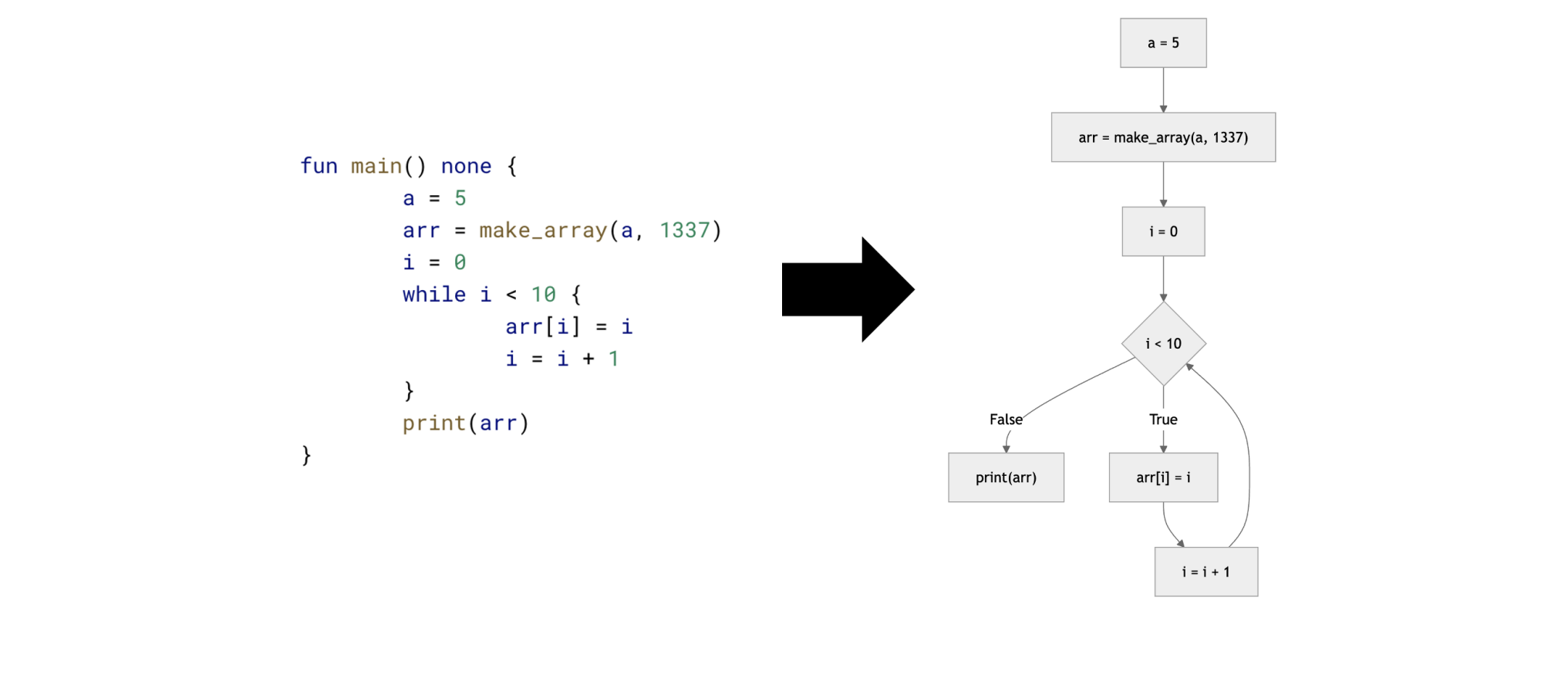
1: Go 코드 → 중간 표현 언어(Imp)

Go AST를 간단한 형태의 Imp 언어로 번역. 분석기는 Imp 코드에 대해 동작하며, 분석기 구현 및 타 언어 이식 시 용이하도록 설계.



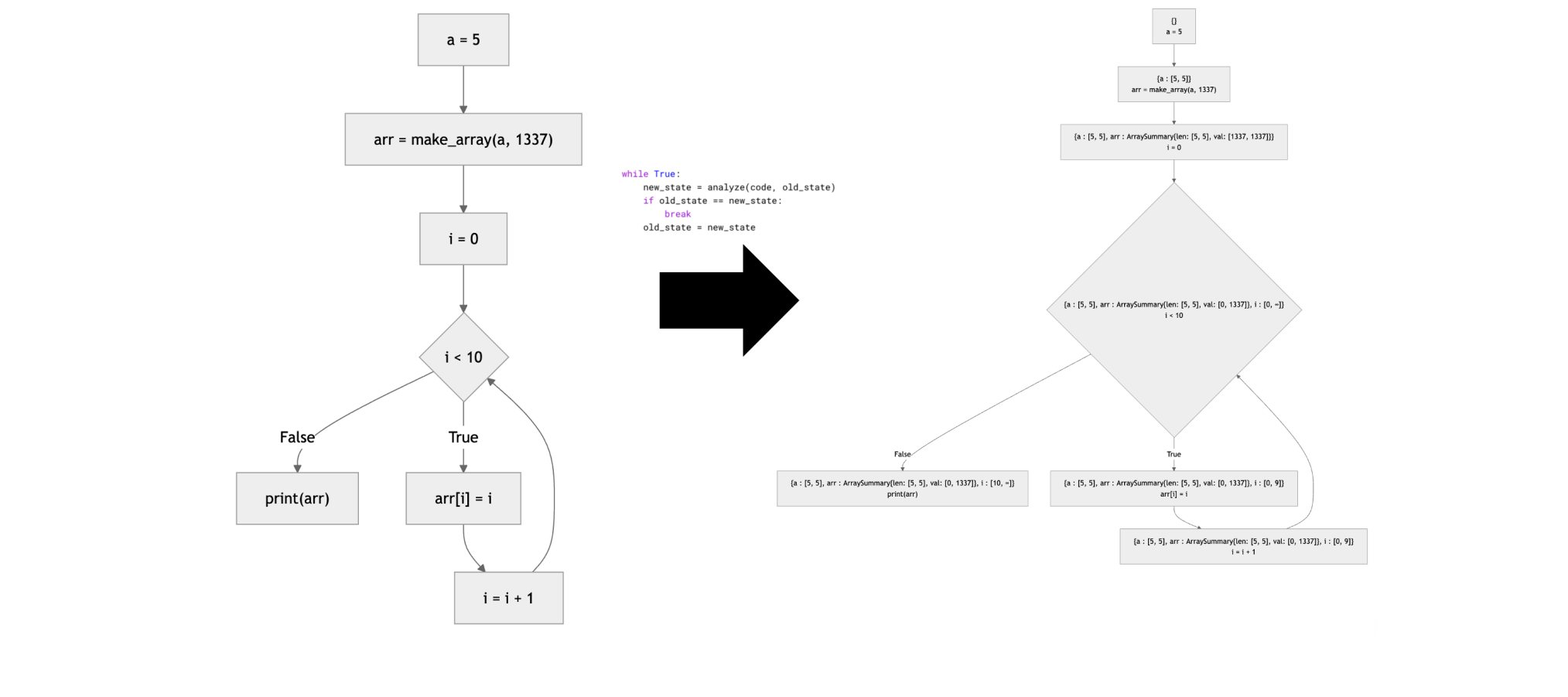
2: Imp → 제어 흐름 그래프(CFG)

프로그램 내 분기, 반복 등의 제어 흐름 단위로 나타내는 제어 흐름 그래프 형태로 표현. 각각의 명령문들은 단일 노드로 표현됨.



3: CFG 위에서 고정점 근사

CFG 노드별 진입 시점에서의 프로그램 상태를 근사. 고정점에 도달할 때까지 반복적으로 요약실행 및 상태 join/widening 수행.



분석 예시: 무한 루프

```
1 func main() {
2   a := 1      a: [1, 1]
3   for a > 0 {  a: [1, 5]
4     a++       a: [1, ∞]
5   }          a: ⊥
6   Print(a)    a: ⊥
7 }
```

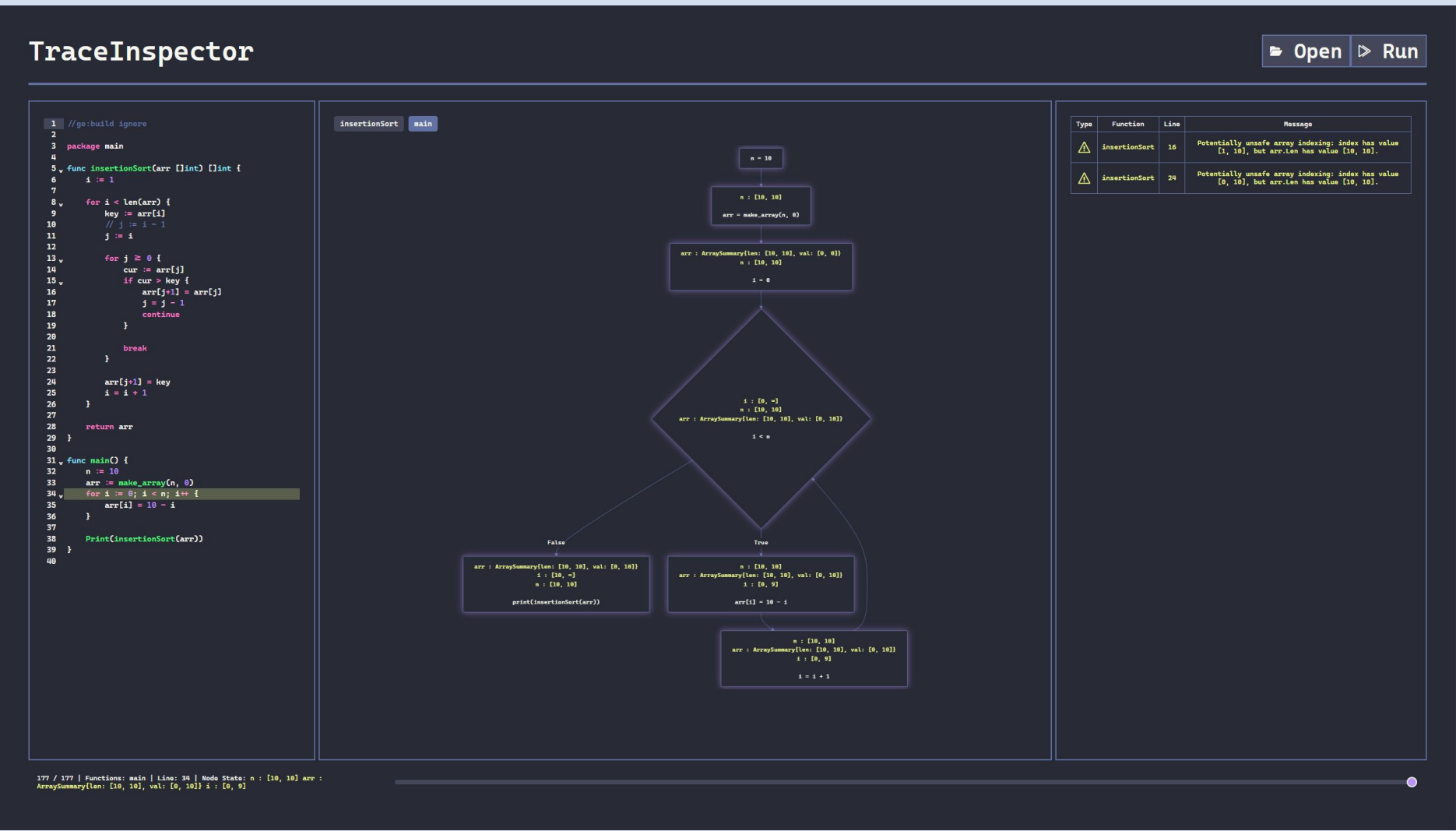
무한히 반복되는 코드의 경우 n 번의 몸통 실행 후, 계산된 상태에 대해 widening 기법을 적용하여 분석이 종료할 수 있도록 유도. 이후 filtering을 적용하여 widening 결과를 정제하여 정확도 손실을 방지함.

분석 예시: 배열 인덱싱 범위

```
1 i := 1          i: [1, 1]
2 for i < len(arr) { i: [1, 10], len(arr): [10, 10]
3   key := arr[i]  i: [1, 9], key: [0, 10]
4   arr[i+1] = arr[i] i: [1, 9], key: [0, 10]
5 Print(i)        i: [10, 10], key: [0, 10]
```

배열의 길이와 인덱스 변수에 대한 요약상태를 계산 후, 안전한 인덱싱 범위인지 확인함.

시각화



하단 슬라이더를 통해 프로그램 실행 상태 변화를 관찰할 수 있음.

한계 및 향후 계획

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Lorem ipsum dolor sit amet, consectetur adipiscing elit.

